

Prevalence and Detection of SQL Antipatterns in Java Projects Using jOOQ for Database Access

Kristo Isberg

Supervisor: Erki Eessaar

Tallinn University of Technology

27.10.2025

What are (SQL) Antipatterns?

- "... a commonly occurring solution to a problem that generates decidedly negative consequences" (Brown et al., 1998)
- "... the most frequent missteps software developers naively make while using SQL" (Karwin, 2022)

Negative consequences? What kind?

- Performance
- Maintainability
- Portability
- Data integrity

An example: *Implicit Columns*

id	name	curr	email	birth_date	uniid
5434	Kristo	IVSM	krisbe@taltech.ee	1998-07-18	krisbe

```
SELECT * FROM student WHERE id = 5434
```

An example: *Implicit Columns*

id	name	curr	email	birth_date	phone
5434	Kristo	IVSM	krisbe@taltech.ee	1998-07-18	57880010

```
var statement = connection.createStatement();
var resultSet = statement.executeQuery("SELECT * FROM student WHERE id = 5434");

if (resultSet.next()) {
    var name = resultSet.getString(2);
    var birthDate = resultSet.getString(5);

    System.out.println(name + " was born on " + birthDate);
}
```

Kristo was born on 1998-07-18

An example: *Implicit Columns*

id (1)	name (2)	curr (3)	email (4)	birth_date (5)	phone (6)
5434	Kristo	IVSM	krisbe@taltech.ee	1998-07-18	57880010

```
var statement = connection.createStatement();
var resultSet = statement.executeQuery("SELECT * FROM student WHERE id = 5434");

if (resultSet.next()) {
    var name = resultSet.getString(2);
    var birthDate = resultSet.getString(5);

    System.out.println(name + " was born on " + birthDate);
}
```

Kristo was born on 1998-07-18

An example: *Implicit Columns*

id (1)	name (2)	email (3)	birth_date (4)	phone (5)	curr_id (6)
5434	Kristo	krisbe@taltech.ee	1998-07-18	57880010	264

```
var statement = connection.createStatement();
var resultSet = statement.executeQuery("SELECT * FROM student WHERE id = 5434");

if (resultSet.next()) {
    var name = resultSet.getString(2);
    var birthDate = resultSet.getString(5);

    System.out.println(name + " was born on " + birthDate);
}
```

Kristo was born on 57880010

An example: *Explicit Columns*

id (1)	name (2)	email (3)	birth_date (4)	phone (5)	curr_id (6)
5434	Kristo	krisbe@taltech.ee	1998-07-18	57880010	264

```
var statement = connection.createStatement();
var resultSet = statement.executeQuery("SELECT name, birth_date FROM student WHERE id = 5434");

if (resultSet.next()) {
    var name = resultSet.getString(1);
    var birthDate = resultSet.getString(2);

    System.out.println(name + " was born on " + birthDate);
}
```

Kristo was born on 1998-07-18

What is jOOQ?

- Open-source Java library for interacting with SQL databases
- Highly popular, especially in industrial projects
- Integrates SQL into Java as a domain-specific language
- Leverages code generation heavily
- Provides strong type safety

Implicit Columns in jOOQ

```
var student = dslContext.select(DSL.asterisk())
    .from(STUDENT)
    .where(STUDENT.ID.eq(5434))
    .fetchOne();

var name = student.getName();
var birthDate = student.getBirthDate();

System.out.println(name + " was born on " + birthDate);
```

SQL Antipatterns in Java

- Prevalent in Java projects
 - *Implicit Columns* affects every 50th query
- Remain unfixed longer than traditional code smells
- Two tools capable of statically detecting SQL Antipatterns in Java
 - None capable of detecting from jOOQ DSL

Research Goal

- Development of tools for detecting SQL antipatterns in database access code using jOOQ
 - Different paradigms: static analysis and language models
- Studying the prevalence of SQL antipatterns in jOOQ

Research Questions: Prevalence

- **RQ1:** How prevalent are SQL antipatterns in existing Java projects that use jOOQ?
- **RQ2:** How can SQL antipatterns manifest in Java projects that use jOOQ?

Research Questions: Detection

- **RQ3:** How accurate are language models in detecting SQL antipatterns in Java projects that use jOOQ?
- **RQ4:** How accurate is traditional static analysis in detecting SQL antipatterns in Java projects that use jOOQ?
- **RQ5:** What is the best way to detect SQL antipatterns in Java projects that use jOOQ?

Thank you for listening!